

Recommended Minimal Architecture

Tomasz Pawłowski 198093, Antoni Sulkowski 197564

1 Realizacja Minimalnej Architektury w kodzie

Zgodnie z wytycznymi projektu, system realizuje zalecaną minimalną architekturę przetwarzania danych. Poniżej przedstawiono poszczególne etapy architektury zaimplementowane w kodzie.

1.1 1. REST API Collector

```
req = urllib.request.Request(API_URL)
req.add_header('Authorization', 'Bearer STUDENT_TOKEN_2026')

try:
    with urllib.request.urlopen(req) as response:
        raw_response = response.read().decode('utf-8')
        weather_data = json.loads(raw_response)
except Exception as e:
    print(f" B d API: {e}")
    return {"statusCode": 500, "body": " B d API"}
```

1.2 2. Raw Storage

Trwały zapis niezmiennych, surowych danych w formacie JSON bezpośrednio do magazynu Amazon S3.

```
file_name = f"raw_data/gdn_01_{timestamp_now}.json"
s3.put_object(
    Bucket=S3_RAW_BUCKET,
    Key=file_name,
    Body=json.dumps(weather_data)
)
```

1.3 3. Validation / Cleaning

Bezpieczne wyodrębnienie wartości numerycznych z wykorzystaniem metody `.get()`. Stanowi to formę walidacji zapobiegającą awariom skryptu w przypadku brakujących kluczy w odpowiedzi API.

```
wind_speed = weather_data.get('wind_speed', 0)
temperature = weather_data.get('temperature', 0)
rain_mm = weather_data.get('rain_mm', 0)
```

1.4 4. Transformation / Feature Engineering

Przekształcanie oraz rzutowanie zmiennych zmiennoprzecinkowych na format tekstowy, w celu zbudowania komunikatów oraz zachowania kompatybilności ze schematem bazy danych.

```
alert_message = " | ".join(alerts_triggered)
station = weather_data.get('station_id', 'GDN_01')

current_temp_str = str(temperature)
current_wind_str = str(wind_speed)
```

1.5 5. Analytics / Expert Rules

Zestaw reguł eksperckich.

```
alerts_triggered = []

if wind_speed > 15.0:
    alerts_triggered.append(f"Silny wiatr: {wind_speed} m/s"
    )

if temperature < 2.0 and rain_mm > 0:
    alerts_triggered.append(f"Ryzyko go oledzi! Temp: {
        temperature}C, Opad: {rain_mm}mm")

if temperature > 35.0:
    alerts_triggered.append(f"Ekstremalny upa : {
        temperature}C")
```

1.6 6. Report / Dashboard / Presentation

Ostateczna prezentacja wyników i dystrybucja alertów realizowana dwutorowo: archiwizacja do raportów w bazie DynamoDB oraz natychmiastowe powiadomienie użytkownika poprzez usługę SNS.

```
table = dynamodb.Table(DYNAMODB_TABLE)
table.put_item(Item={
    'alert_id': str(uuid.uuid4()),
    'timestamp': timestamp_now,
    'station_id': station,
    'triggered_rules': alert_message,
    'current_temp': current_temp_str,
```

```

        'current_wind': current_wind_str
    })

    email_body = (
        f"OSTRZE ENIE POGODOWE DLA STACJI {station}\n"
        f"Czas: {timestamp_now}\n\n"
        f"Wykryte zagro enia:\n- {chr(10) + '- '}.join(
            alerts_triggered)}\n"
    )
    sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Message=email_body,
        Subject=f"Alert Pogodowy - {station}"
    )

```